



EnterpriseDataManager

Пълен Технически Одит на Проекта

Тип документ: Анализ на покритие и план за изпълнение

Дата: 19 Април 2026 г.

Версия: 1.0

Платформа: ASP.NET Core 8 · PostgreSQL · Docker

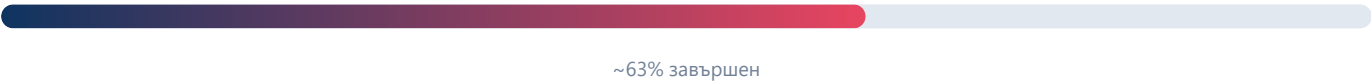
ПОВЕРИТЕЛНО

Съдържание

1. Контекст и изисквания	3
2. Общо покритие на проекта	3
3. Напълно имплементирани функционалности	4
4. Критични пропуски и stubs	5
5. Липсващи функционалности по спецификация	6
6. Строг план за изпълнение	7
7. Цени за внедряване и поддръжка	9
8. Обобщение	11

Обща оценка на завършеност

Проектът е на приблизително **60–65% завършеност**. Архитектурната основа е добре изградена, но критични компоненти са заглушени (stubbed) или липсват изцяло.



⚠ Важно: Проектът не е готов за продукционно внедряване в текущото си състояние. Критичните stubs в ArchivalJobScheduler, RetentionPolicyEnforcer и HealthCheckMonitor блокират реалната функционалност.

1. Контекст и изисквания

Документът анализира проекта спрямо следните две задължителни системи:

Система за архивиране на информация

Изискване	Статус
Анализ и изготвяне на план за архивиране	✓ Покрито
Автоматично архивиране от сървъри	⚡ Частично
Автоматично архивиране от работни станции	✗ Липсва
Автоматично архивиране от мобилни устройства	✗ Липсва
Вградена защита от ransomware	✓ Покрито
Интеграция с централизирано управление на достъпа	✓ Покрито (LDAP + OIDC)
Процедури за възстановяване от архив	✓ Покрито
Минимум едно пробно възстановяване	✗ Липсва (UI wizard)
Виртуален/физически сървър за съхранение	✓ Docker + Local/Azure/S3
Лентово устройство/робот за архивиране	⚡ Частично (Mock + CLI)

Система за управление на съхранението и споделянето

Изискване	Статус
Централизирано управление на достъпа на служителите	⚡ Частично (Auth налично, ACL липсва)
Интеграция с останалите системи на организацията	✓ LDAP + OIDC + MFA
Журнален запис на действията на потребителите	✓ Покрито
Виртуален/физически сървър	✓ Docker Compose

2. Напълно имплементирани функционалности

Функционалност	Компонент	Статус
Създаване и управление на план за архивиране	ArchivePlanService	✓ Реален код
Архивни задачи с изпълнение и проследяване	ArchivalService, ArchiveJob	✓ Реален код
AES-256-GCM криптиране с PBKDF2	AesEncryptionService	✓ Реален код
Ransomware детекция (аномалии, WORM)	AnomalyDetector, WormSimulator, ImmutableStorageService	✓ Реален код
LDAP / Active Directory интеграция	LdapConnector	✓ Реален код
OIDC / OAuth2 / SSO интеграция	OidcConnector	✓ Реален код
TOTP MFA (Google Authenticator)	TotpMfaProvider	✓ Реален код
JWT автентикация + refresh token	IdentityService	✓ Реален код
Задачи за възстановяване + SHA-256 верификация	RecoveryService	✓ Реален код
Политики за съхранение + правна забрана	PolicyEngine, RetentionPolicy	✓ Реален код
Журнален запис (Audit Log)	AuditService, AuditRecord	✓ Реален код
Локална файлова система (path traversal защита)	LocalFilesystemProvider	✓ Реален код
Azure Blob Storage	AzureBlobProvider	✓ Реален код
S3/S3-compatible хранилище	S3CompatibleProvider	✓ Реален код
EF Core + PostgreSQL + миграции	EnterpriseDataManagerDbContext	✓ Реален код
Security headers (HSTS, CSP, CORS)	SecurityHeadersMiddleware	✓ Реален код
Структурирано логване (Serilog)	StructuredLogEnricher	✓ Реален код
Docker Compose (postgres + web)	docker-compose.yml	✓ Реален код
Локализация (BG + EN)	Resources/, LanguageController	✓ Реален код
Domain events, Unit of Work, Repository pattern	Core layer	✓ Реален код

✓ **Солидна основа:** Всички основни инфраструктурни компоненти са реално имплементирани — криптиране, идентичност, одит, съхранение и domain логика. Проблемите са в нивото на оркестрация и липсващи крайни функционалности.

3. Критични пропуски и Stubs

Следните компоненти са налице в кода, но съдържат **фиктивна/заглушена имплементация**, която трябва да бъде заменена преди продукционно пускане:

Компонент	Файл	Проблем	Риск
Scheduler не изпълнява реални задачи	ArchivalJobScheduler.cs · RunJobAsync()	Връща фиктивни метрики; реалният IArchivalService не се извиква	КРИТИЧЕН
Retention enforcer — твърдо кодирани данни	RetentionPolicyEnforcer.cs · GetAffectedItemsAsync()	Връща List{"item1", "item2"} — никога не пита базата данни	КРИТИЧЕН
Health checks са симулирани	HealthCheckMonitor.cs	DB, cache, мрежови проверки връщат твърдо кодирано true	КРИТИЧЕН
MFA не е персистентно	InMemoryMfaStateStore	Всички MFA сесии се губят при рестарт на приложението	ВИСОК
SIEM Forwarders непотвърдени	CefForwarder.cs, SyslogForwarder.cs	Вероятно stubs — SIEM интеграцията е под въпрос	ВИСОК
Лентово устройство изисква mt CLI	TapeDeviceAdapter.cs	Срива се на Windows без tape driver инструменти	СРЕДЕН

4. Липсващи функционалности по спецификация

Функционалност	Изисква се от	Статус
Агент за работни станции — Windows Service за автоматично архивиране	Система за архивиране	✗ Напълно липсва
Мобилно архивиране — REST API или MDM (Intune) конектор	Система за архивиране	✗ Напълно липсва
Тестово пробно възстановяване — UI wizard в симулационен режим	Система за архивиране	✗ Напълно липсва
ACL права на ниво файл/ресурс — FilePermission entity + управление	Система за управление на съхранение	✗ Напълно липсва
Споделяне на файлове — генериране на споделени връзки с права	Система за управление на съхранение	✗ Напълно липсва
Табло с реални метрики — Dashboard с живи данни	Обща платформа	⚡ Scaffold само
Модул за отчети — PDF/Excel експорт на одит и история	Обща платформа	⚡ Scaffold само

5. Строг план за изпълнение

1 Фаза 1 — Поправяне на критичните Stubs | 2–3 седмици

1. Свързване на `ArchivalJobScheduler.RunJobAsync()` → извикване на реалния `IArchivalService.CreateAndStartJobAsync()`
2. Поправяне на `RetentionPolicyEnforcer.GetAffectedItemsAsync()` → заявка към `IArchiveItemRepository` с филтър по план/политика
3. Замяна на всички твърдо кодирани проверки в `HealthCheckMonitor` с реални: `DbContext.Database.CanConnectAsync()`, `IStorageProviderAdapter.HealthCheckAsync()` и т.н.
4. Имплементиране на DB-базиран `IMfaStateStore` (EF Core таблица: `MfaStates`)
5. Верификация и завършване на `CefForwarder / SyslogForwarder` — реално TCP/UDP syslog предаване

2 Фаза 2 — Персистентност и надеждност | 1–2 седмици

1. Добавяне на персистентно проследяване на `LastRun` в `ArchivalJobScheduler` (запис в БД)
2. Добавяне на поддръжка на Windows Driver за `TapeDeviceAdapter` или интеграция с vendor SDK (LTO Ultrium, HP StoreEver)
3. Добавяне на retry механизъм + опашка за неуспешни архивни задачи
4. Свързване на `EmailService / NotificationService` към събитията за завършване и неуспех на задачи

3 Фаза 3 — Липсващи функционалности по спецификация | 3–4 седмици

1. **Агент за работни станции** — лека Windows Service (.NET Worker), която извиква Archive API-то автоматично по разписание
2. **Архивиране от мобилни устройства** — REST API endpoint приемащ качване на файлове от iOS/Android или интеграция с MDM (Microsoft Intune) webhook
3. **Тестово пробно възстановяване** — UI wizard, който клонира реален `RecoveryJob` в симулационен режим, отчита успех/неуспех без презаписване на продукционни данни
4. **Споделяне на файлове и ACL** — добавяне на entity `FilePermission` (`UserId`, `ResourceId`, `AccessLevel`), `IShareService`, интерфейс за управление

4 Фаза 4 — Завършване на интерфейса | 2–3 седмици

1. Табло — свързване на `DashboardApiController` с реални агрегирани заявки (задачи днес, използвано хранилище, последни одит събития, брой аномалии)
2. Отчети — PDF/Excel експорт на одит логове, история на задачи, използване на хранилище (FastReport или NPOI)
3. Интерфейс за управление на доставчици на съхранение (в момента само API)
4. Одит лог UI — пълен екран за търсене, филтриране и експорт

5 Фаза 5 — Подготовка за продукция | 1–2 седмици

1. Продукционен Docker Compose профил (secrets, TLS терминация чрез nginx/traefik)
2. CI/CD pipeline — завършване на GitHub Actions: build → test → SAST сканиране → Docker push → deploy
3. Натоварващо/стрес тестване на архивните задачи (симулация с 10 000 файла)
4. Контролен списък за penetration тест спрямо OWASP Top 10

6. Цени за внедряване и поддръжка

* Всички цени са приблизителни оценки в лева (BGN), базирани на пазарни нива в България.

Еднократни разходи за завършване на разработката

Фаза	Усилие	Минимална цена	Максимална цена
Фаза 1 — Поправка на stubs	2–3 седмици × 1 старши разработчик	5 800 лв.	9 700 лв.
Фаза 2 — Надеждност и персистентност	1–2 седмици	2 900 лв.	5 800 лв.
Фаза 3 — Липсващи функционалности	3–4 седмици	9 700 лв.	17 500 лв.
Фаза 4 — Завършване на UI	2–3 седмици	5 800 лв.	11 700 лв.
Фаза 5 — Втвърдяване + QA	1–2 седмици	3 900 лв.	7 800 лв.
ОБЩО оставаща разработка	~3–4 месеца	28 100 лв.	52 500 лв.

Инфраструктура — Вариант А: Облак (препоръчително)

Елемент	Описание	Месечна цена
Виртуален сървър	4 vCPU, 16 GB RAM (Azure/AWS)	230 – 390 лв.
Управлявана PostgreSQL БД	Azure Database / RDS	155 – 290 лв.
Blob/S3 хранилище	1–5 TB данни	100 – 390 лв.
Load balancer + TLS	HTTPS терминация	60 – 115 лв.
Мониторинг	Grafana Cloud / Azure Monitor	0 – 100 лв.
ОБЩО / месец		545 – 1 285 лв.

Инфраструктура — Вариант Б: На място (при изискване за лентово устройство)

Елемент	Описание	Еднократна цена
Виртуален сървър	Dell/HPE, 32 GB RAM	5 800 – 11 700 лв.
LTO-8 лентово устройство	HPE StoreEver или еквивалент	4 900 – 9 700 лв.
Лентови касети	20 броя LTO-8 (по 12 TB)	1 200 – 2 300 лв.
UPS + стойка (rack)	Физическа инфраструктура	1 600 – 2 900 лв.
Настройка на ОС и платформа	Windows Server или Linux	1 000 – 2 000 лв.
ОБЩО еднократно		14 500 – 28 600 лв.

Годишни разходи за поддръжка

Елемент	Описание	Цена / Година
Облачна инфраструктура	12 месеца × месечна цена	6 500 – 15 400 лв.
Актуализации за сигурност и крёпки	Месечни patch cycles	2 000 – 4 000 лв.
Подновяване на лицензи	Vendor SDKs, ако е приложимо	0 – 1 000 лв.
Договор за поддръжка	Разработчик при нужда, 8 ч/месец	5 800 – 11 700 лв.
Годишно тестово пробно възстановяване	Drill + отчет до ръководство	1 000 – 2 000 лв.
ОБЩО годишна поддръжка		15 300 – 34 100 лв.

7. Обобщение

Категория	Минимална оценка	Максимална оценка
Оставаща разработка	28 100 лв.	52 500 лв.
Инфраструктура — облак (еднократно)	0 лв.	3 900 лв.
Инфраструктура — на място (еднократно)	14 500 лв.	28 600 лв.
Годишна поддръжка	15 300 лв.	34 100 лв.

Общ извод: Проектът разполага с **добре архитектирана основа (~60–65% завършен)**. Всички критични инфраструктурни компоненти — криптиране, идентичност, одит, съхранение — са реално имплементирани. Най-спешните задачи са поправката на стабовете в background job слоя, добавянето на агенти за работни станции и мобилни устройства, и имплементацията на ACL/споделяне — всички изрично изисквани от спецификацията.

Препоръчителна следваща стъпка: Стартирайте с Фаза 1 (2–3 седмици), за да направите системата функционално коректна. Само след нея приложението може да се счита за минимално работещо (MVP) за тестова среда.

